

New Taipei City Housing Price Prediction

Begimai Bolotbekova

2024-06-01

- Introduction
- 1 Data Exploration
- 2 Data Preprocessing
- 3 Model Training and Evaluation
 - 3.1 Multiple Linear Regression (MLR)
 - 3.2 Random Forest
 - 3.3 XGBoost
- 4 Model Comparison

Introduction

This project predicts **price per square meter (sqmPrice)** for apartments in **New Taipei City, Taiwan** using three machine learning models: MLR, Random Forest, and XGBoost.

This project aims to predict the price per square meter (sqmPrice) for residential transactions in New Taipei City, Taiwan. The project uses a dataset covering housing sales transactions from 2018 to 2023. To find the most accurate predictive power, three distinct modeling approaches: Multiple Linear Regression (MLR), Random Forest, and XGBoost were evaluated.

1 Data Exploration

```
housing <- read_csv("data/RAW_new_taipei_housing_transactions_2018_2023.csv")
str(housing)
```

```

## spc_tbl_ [53,682 × 32] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ district: chr [1:53682] "Bali" "Bali" "Bali" "Bali" ...
## $ rps01   : chr [1:53682] " " " " " " ( + )+ " " ( + )+ " ...
## $ rps02   : chr [1:53682] " 104 " " 1711 " " Bali " " Bali
" ...
## $ rps03   : num [1:53682] 2199 32.3 28.2 10.1 86.3 ...
## $ rps04   : chr [1:53682] " : " " : " " " " " ...
## $ rps05   : chr [1:53682] NA NA NA NA ...
## $ rps06   : chr [1:53682] NA NA NA NA ...
## $ rps07   : num [1:53682] 1110215 1110215 1110207 1110207 1110220 ...
## $ rps08   : chr [1:53682] " 1 0 0" " 6 0 0" " 1 1 1" " 1 1 1" ...
## $ rps09   : chr [1:53682] NA NA " " " " " ...
## $ rps10   : chr [1:53682] NA NA " " " " " ...
## $ rps11   : chr [1:53682] " " " " " (11 )" " (11 )" ...
## $ rps12   : chr [1:53682] NA NA " " " " " ...
## $ rps13   : chr [1:53682] NA NA " " " " " ...
## $ rps14   : num [1:53682] NA NA 1090818 1100416 970603 ...
## $ rps15   : num [1:53682] 0 0 138 101 210 ...
## $ rps16   : num [1:53682] 0 0 3 2 3 4 0 3 4 0 ...
## $ rps17   : num [1:53682] 0 0 2 2 3 2 0 2 2 0 ...
## $ rps18   : num [1:53682] 0 0 2 1 3 2 0 2 2 0 ...
## $ rps19   : chr [1:53682] " " " " " " " " " ...
## $ rps20   : chr [1:53682] " " " " " " " " " ...
## $ rps21   : num [1:53682] 10680000 184484 10120000 6900000 14900000 ...
## $ rps22   : num [1:53682] 4857 5705 82018 68007 71067 ...
## $ rps23   : chr [1:53682] NA NA " " " " " ...
## $ rps24   : num [1:53682] 0 0 34.3 32.9 0 ...
## $ rps25   : num [1:53682] 0 0 1650000 0 0 0 0 0 0 800000 ...
## $ rps26   : chr [1:53682] " " NA NA NA ...
## $ rps27   : chr [1:53682] "RP00MLRKKHHGFEF08EA" "RPTRMLLLKHHGFEF38EA" "RPPNMLQLKHHGFEF97DA" "RPOPMLMJ
LHHGFEF57DA" ...
## $ rps28   : num [1:53682] 0 0 62.1 40.2 127.5 ...
## $ rps29   : num [1:53682] 0 0 0 2.29 0 ...
## $ rps30   : num [1:53682] 0 0 7.09 4.5 21 ...
## $ rps31   : chr [1:53682] " " " " " " " " " ...

```

```
## - attr(*, "spec")=
## .. cols(
## ..   district = col_character(),
## ..   rps01 = col_character(),
## ..   rps02 = col_character(),
## ..   rps03 = col_double(),
## ..   rps04 = col_character(),
## ..   rps05 = col_character(),
## ..   rps06 = col_character(),
## ..   rps07 = col_double(),
## ..   rps08 = col_character(),
## ..   rps09 = col_character(),
## ..   rps10 = col_character(),
## ..   rps11 = col_character(),
## ..   rps12 = col_character(),
## ..   rps13 = col_character(),
## ..   rps14 = col_double(),
## ..   rps15 = col_double(),
## ..   rps16 = col_double(),
## ..   rps17 = col_double(),
## ..   rps18 = col_double(),
## ..   rps19 = col_character(),
## ..   rps20 = col_character(),
## ..   rps21 = col_double(),
## ..   rps22 = col_double(),
## ..   rps23 = col_character(),
## ..   rps24 = col_double(),
## ..   rps25 = col_double(),
## ..   rps26 = col_character(),
## ..   rps27 = col_character(),
## ..   rps28 = col_double(),
## ..   rps29 = col_double(),
## ..   rps30 = col_double(),
## ..   rps31 = col_character()
```

```
##    .. )  
##  - attr(*, "problems")=<externalptr>
```

```
summary(housing)
```

##	district	rps01	rps02	rps03
##	Length:53682	Length:53682	Length:53682	Min. : 0.00
##	Class :character	Class :character	Class :character	1st Qu.: 13.26
##	Mode :character	Mode :character	Mode :character	Median : 21.11
##				Mean : 104.87
##				3rd Qu.: 31.37
##				Max. :46668.90
##				
##	rps04	rps05	rps06	rps07
##	Length:53682	Length:53682	Length:53682	Min. : 110801
##	Class :character	Class :character	Class :character	1st Qu.:1101128
##	Mode :character	Mode :character	Mode :character	Median :1110318
##				Mean :1105381
##				3rd Qu.:1110717
##				Max. :1121114
##				
##	rps08	rps09	rps10	rps11
##	Length:53682	Length:53682	Length:53682	Length:53682
##	Class :character	Class :character	Class :character	Class :character
##	Mode :character	Mode :character	Mode :character	Mode :character
##				
##				
##				
##				
##	rps12	rps13	rps14	rps15
##	Length:53682	Length:53682	Min. : 100000	Min. : 0.00
##	Class :character	Class :character	1st Qu.: 820610	1st Qu.: 67.93
##	Mode :character	Mode :character	Median : 961130	Median : 105.29
##			Mean : 933224	Mean : 118.47
##			3rd Qu.:1100427	3rd Qu.: 149.54
##			Max. :1120725	Max. :29007.88
##			NA's :8812	
##				
##	rps16	rps17	rps18	rps19
##	Min. : 0.000	Min. : 0.000	Min. : 0.000	Length:53682
##	1st Qu.: 1.000	1st Qu.: 1.000	1st Qu.: 1.000	Class :character

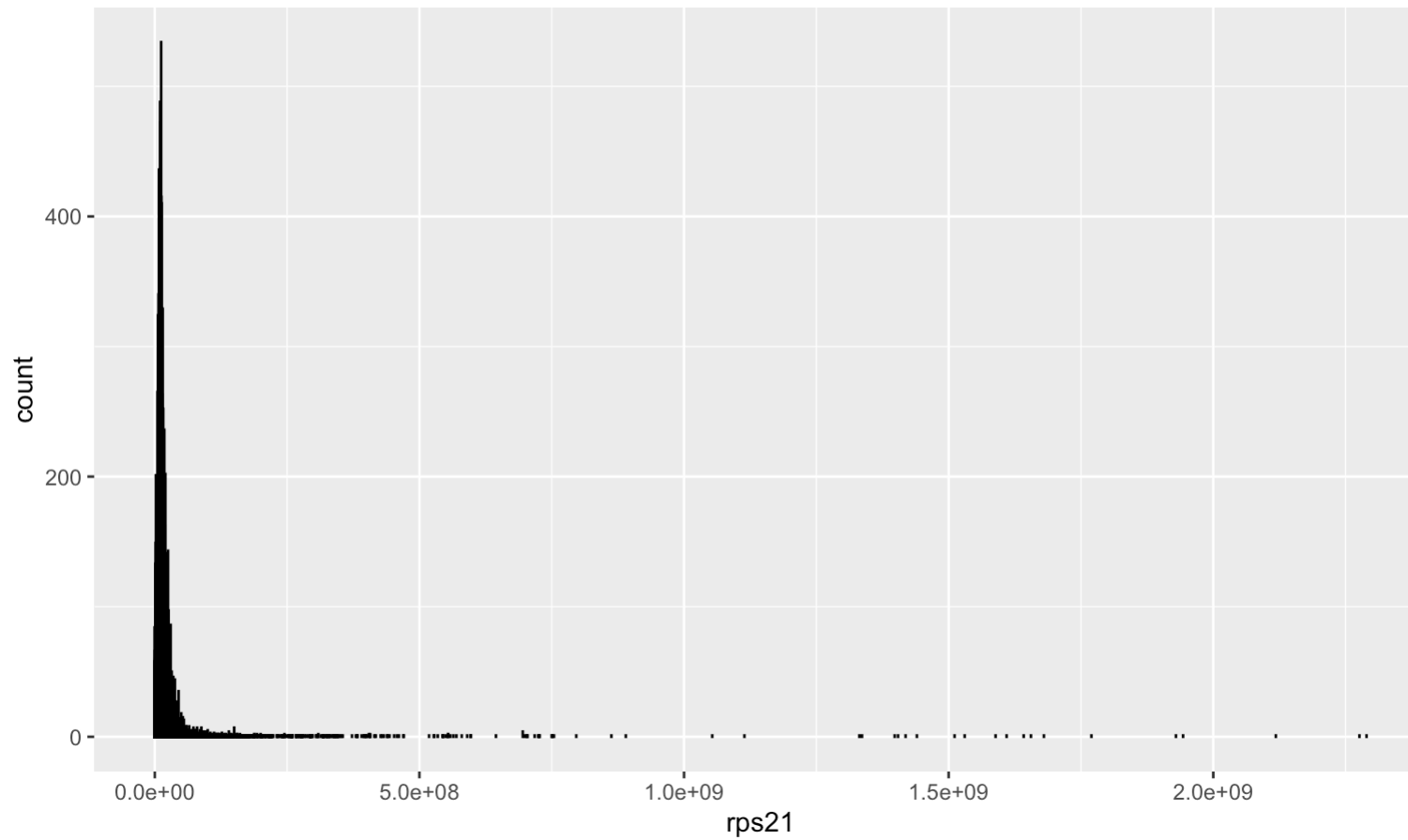
```
## Median : 2.000 Median : 2.000 Median : 1.000 Mode :character
## Mean : 2.138 Mean : 1.388 Mean : 1.338
## 3rd Qu.: 3.000 3rd Qu.: 2.000 3rd Qu.: 2.000
## Max. :77.000 Max. :20.000 Max. :101.000
##
##      rps20      rps21      rps22      rps23
## Length:53682 Min. :2.120e+02 Min. : 0 Length:53682
## Class :character 1st Qu.:7.300e+06 1st Qu.: 79901 Class :character
## Mode :character Median :1.155e+07 Median : 111412 Mode :character
## Mean :1.601e+07 Mean : 114526
## 3rd Qu.:1.727e+07 3rd Qu.: 144510
## Max. :2.290e+09 Max. :7546326
## NA's :1515
##      rps24      rps25      rps26      rps27
## Min. : 0.00 Min. : 0 Length:53682 Length:53682
## 1st Qu.: 0.00 1st Qu.: 0 Class :character Class :character
## Median : 0.00 Median : 0 Mode :character Mode :character
## Mean : 29.17 Mean : 649815
## 3rd Qu.: 30.46 3rd Qu.: 1400000
## Max. :800000.00 Max. :126160000
##
##      rps28      rps29      rps30      rps31
## Min. : 0.00 Min. : 0.000 Min. : 0.00 Length:53682
## 1st Qu.: 41.04 1st Qu.: 0.000 1st Qu.: 0.00 Class :character
## Median : 66.10 Median : 0.000 Median : 5.12 Mode :character
## Mean : 71.87 Mean : 2.222 Mean : 5.93
## 3rd Qu.: 87.12 3rd Qu.: 2.880 3rd Qu.: 9.10
## Max. :17662.34 Max. :966.730 Max. :285.95
##
```

```
sapply(housing, function(x) sum(is.na(x)))
```

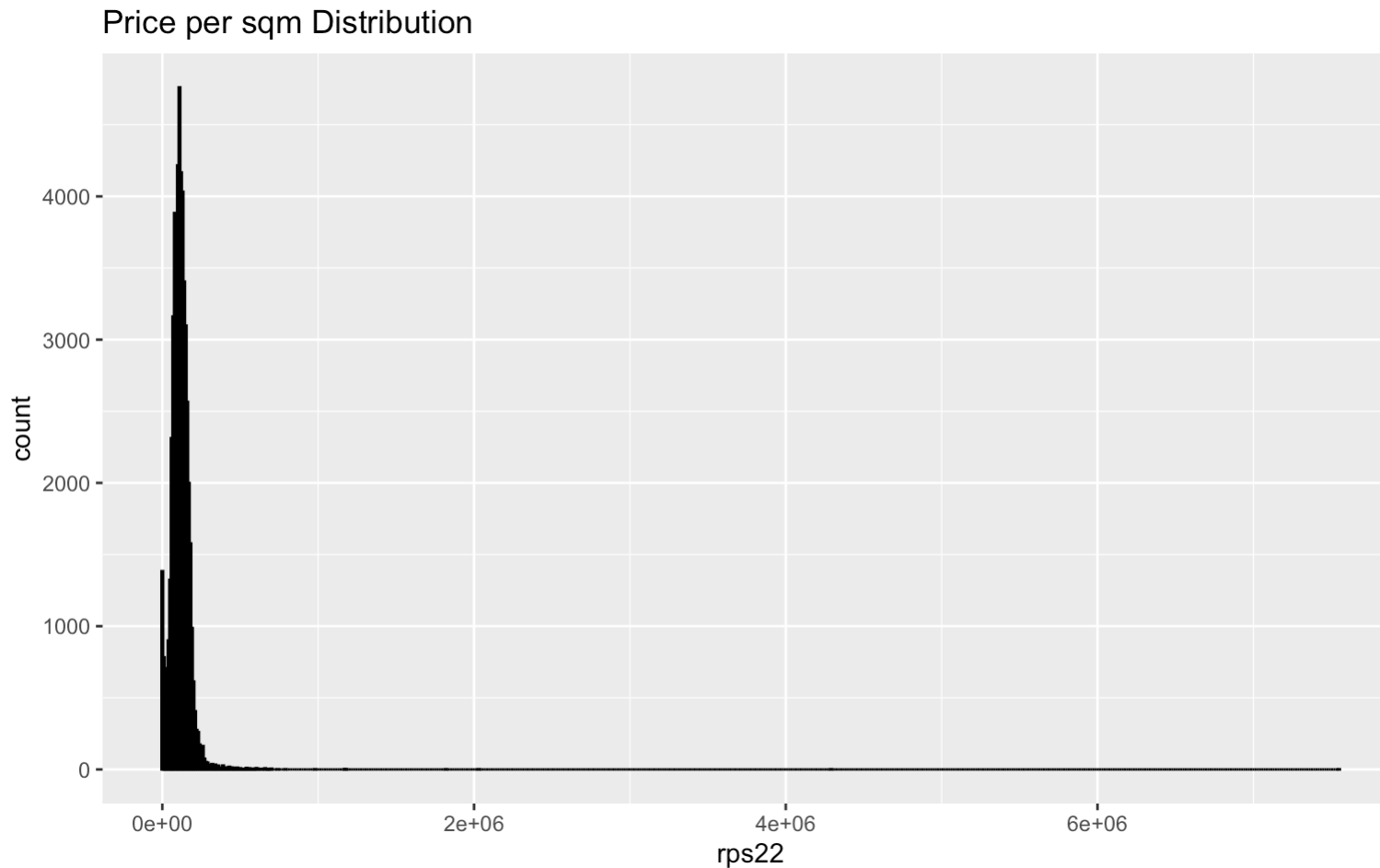
##	district	rps01	rps02	rps03	rps04	rps05	rps06	rps07
##	0	0	0	0	4972	49848	49861	0
##	rps08	rps09	rps10	rps11	rps12	rps13	rps14	rps15
##	0	7442	7445	0	7454	7443	8812	0
##	rps16	rps17	rps18	rps19	rps20	rps21	rps22	rps23
##	0	0	0	0	0	0	1515	28802
##	rps24	rps25	rps26	rps27	rps28	rps29	rps30	rps31
##	0	0	22478	0	0	0	0	0

```
ggplot(housing, aes(x = rps21)) +
  geom_histogram(binwidth = 10000, fill="skyblue", color="black") +
  labs(title = "Total Price Distribution")
```

Total Price Distribution



```
ggplot(housing, aes(x = rps22)) +  
  geom_histogram(binwidth = 10000, fill="lightgreen", color="black") +  
  labs(title = "Price per sqm Distribution")
```

2 Data Preprocessing

In this step, irrelevant columns and extreme outliers, which were identified via the IQR method, were removed. Missing values in urban zoning and parking categories were handled through logical imputation and recoding. Date formats were standardized from the Taiwanese calendar to Gregorian, and structural data, such as Chinese floor notations were converted into numeric values. Key features were engineered by calculating building age and transforming binary indicators into logical types. Finally, logarithmic transformations were applied to skewed numerical variables, such as price and area, to stabilize variance and improve model performance.

```

housing <- read_csv("data/RAW_new_taipei_housing_transactions_2018_2023.csv")
housing <- housing %>% select(-rps01, -rps12)

missing_rows_rps04 <- which(is.na(housing$rps04))
housing$rps04[missing_rows_rps04[housing$rps05[missing_rows_rps04] != "NA"]] <- "non-urban"
housing$rps04[is.na(housing$rps04)] <- names(sort(table(housing$rps04), decreasing = TRUE))[1]
housing <- housing %>% select(-rps05, -rps06)

taiwanese_to_date <- function(x) {
  year <- x %/% 10000 + 1911
  month <- (x %% 10000) %/% 100
  day <- x %% 100
  ymd(paste(year, month, day, sep = "-"))
}

housing <- housing %>% mutate(rps14 = taiwanese_to_date(rps14), rps07 = taiwanese_to_date(rps07)) %>% filter(!is.na(rps14), !is.na(rps10))

housing$rps23[is.na(housing$rps23)] <- "no parking"

cols <- c("rps03", "rps15", "rps21", "rps22", "rps24", "rps25", "rps30")
for (col in cols) {
  q1 <- quantile(housing[[col]], 0.25, na.rm = TRUE)
  q3 <- quantile(housing[[col]], 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  lower <- q1 - 1.5*iqr
  upper <- q3 + 1.5*iqr
  housing <- housing %>% filter(is.na(.data[[col]]) | (.data[[col]] >= lower & .data[[col]] <= upper))
}

housing$age <- round(as.numeric(difftime(housing$rps07, housing$rps14, units="days")) / 365.25)

housing <- housing %>% rename(
  district = district,
  address = rps02,

```

```

landArea = rps03,
landUseZoning = rps04,
transactionDate = rps07,
package = rps08,
apartmentFloor = rps09,
totalFloors = rps10,
buildingType = rps11,
buildingMaterials = rps13,
constructionDate = rps14,
buildingArea = rps15,
room = rps16,
livingroom = rps17,
bathroom = rps18,
divisionWall = rps19,
management = rps20,
totalPrice = rps21,
sqmPrice = rps22,
parkingCategory = rps23,
parkingArea = rps24,
parkingPrice = rps25,
remarks = rps26,
transactionNumber = rps27,
mainBuildingArea = rps28,
annexBuildingArea = rps29,
balconyArea = rps30,
elevator = rps31
)

chinese_to_int <- function(x) {
  map <- c(" "=0," "=1," "=2," "=3," "=4," "=5," "=6," "=7," "=8," "=9," "=10)
  sapply(x, function(val) {
    if(is.na(val)) return(NA_real_)
    floors <- str_split(val, "[, ]")[[1]]
    floors <- str_remove_all(floors, "[ ]")
    numeric_floors <- sapply(floors, function(f) { f <- str_trim(f); map[f] })
  })
}

```

```

    numeric_floors <- numeric_floors[!is.na(numeric_floors)]
    if(length(numeric_floors)!=1) return(NA_real_)
    numeric_floors
  })
}

char_to_logical <- function(x) if_else(x==" ", TRUE, FALSE)

housing <- housing %>% mutate(
  management = char_to_logical(management),
  elevator = char_to_logical(elevator),
  totalPriceL = log(totalPrice),
  balconyAreaL = log(balconyArea),
  buildingAreaL = log(buildingArea),
  parkingAreaL = log(parkingArea),
  parkingPriceL = log(parkingPrice),
  sqmPriceL = log(sqmPrice),
  totalFloors = chinese_to_int(totalFloors),
  apartmentFloor = chinese_to_int(apartmentFloor)
)

write_csv(housing, "data/PROCESSED_new_taipei_housing_transactions_2018_2023.csv")

```

3 Model Training and Evaluation

This section evaluates three models: Multiple Linear Regression (MLR), Random Forest (RF), and XGBoost.

Each model is trained on 80% of the data and evaluated on the remaining 20% to estimate out-of-sample performance.

3.1 Multiple Linear Regression (MLR)

```
housing <- read_csv("data/PROCESSED_new_taipei_housing_transactions_2018_2023.csv")

housing_X <- housing %>%
  select(
    sqmPriceL,
    buildingAreaL,
    age,
    totalFloors,
    district,
    buildingType,
    parkingAreaL,
    balconyAreaL,
    landUseZoning,
    parkingCategory
  ) %>%
  filter(if_all(everything(), ~ !is.na(.) & !is.nan(.) & !is.infinite())) %>%
  droplevels()

smp_size <- floor(0.8 * nrow(housing_X))
train_ind <- sample(seq_len(nrow(housing_X)), size = smp_size)

train <- housing_X[train_ind, ]
test  <- housing_X[-train_ind, ]

Y_test <- test$sqmPriceL
test$sqmPriceL <- NULL

linearModel <- lm(sqmPriceL ~ ., data = train)
summary(linearModel)
```

```
##
## Call:
## lm(formula = sqmPriceL ~ ., data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.98086 -0.11358  0.00492  0.12489  0.57225
##
## Coefficients:
##              Estimate Std. Error t value
## (Intercept)    10.7488783   0.1858180   57.846
## buildingAreaL     0.0358963   0.0374724    0.958
## age             -0.0043976   0.0007634   -5.761
## totalFloors       0.0408890   0.0047446    8.618
## districtBanqiao    0.6215628   0.0575436   10.802
## districtJinshan   -0.2618953   0.0739411   -3.542
## districtLuzhou     0.6008929   0.0545378   11.018
## districtRuifang   -0.6433843   0.1637953   -3.928
## districtSanchong    0.5996515   0.0533195   11.246
## districtSanxia     0.3848472   0.0615648    6.251
## districtSanzhi    -0.7692274   0.1190462   -6.462
## districtShinkeng    0.2322147   0.0889726    2.610
## districtShulin     0.1901990   0.0518279    3.670
## districtTaishan    0.4369644   0.0616332    7.090
## districtTamsui     0.0667084   0.0525343    1.270
## districtTucheng    0.4127171   0.0658406    6.268
## districtWugu       0.2669285   0.0542136    4.924
## districtXindian    0.6356821   0.0518456   12.261
## districtXinzhuang  0.5659366   0.0549253   10.304
## districtXizhi      0.4609337   0.0551372    8.360
## districtYingge     0.0195259   0.0535719    0.364
## districtYonghe     0.9470883   0.0748904   12.646
## districtZhonghe    0.6300681   0.0576889   10.922
## buildingType      0.2481815   0.2165982    1.146
## buildingType (10 ) 0.2330585   0.0524701    4.442
```

## parkingAreaL		-0.0256973	0.0199179	-1.290
## balconyAreaL		0.0003728	0.0156321	0.024
## landUseZoning		-0.0957304	0.0295963	-3.235
## landUseZoning		-0.0543553	0.0533759	-1.018
## landUseZoning		-0.2574882	0.0415956	-6.190
## landUseZoning		0.0151533	0.0766213	0.198
## landUseZoning	:	-0.0128716	0.0605390	-0.213
## landUseZoning	: ()	0.5474369	0.2108286	2.597
## landUseZoning	:	-0.5667021	0.1262464	-4.489
## landUseZoning	:	0.3403453	0.2117962	1.607
## landUseZoning	:	-0.4691926	0.0791622	-5.927
## landUseZoning	:	-0.1556408	0.1585832	-0.981
## landUseZoning	:	-0.0800423	0.1127443	-0.710
## parkingCategory		-0.0406942	0.0831586	-0.489
## parkingCategory		-0.0323767	0.0791185	-0.409
## parkingCategory		-0.0525026	0.0737143	-0.712
## parkingCategory		-0.1391133	0.0711546	-1.955
## parkingCategory		-0.1439518	0.0735573	-1.957
## parkingCategory		-0.0211871	0.0794590	-0.267
##		Pr(> t)		
## (Intercept)		< 2e-16 ***		
## buildingAreaL		0.338293		
## age		1.07e-08 ***		
## totalFloors		< 2e-16 ***		
## districtBanqiao		< 2e-16 ***		
## districtJinshan		0.000413 ***		
## districtLuzhou		< 2e-16 ***		
## districtRuifang		9.07e-05 ***		
## districtSanchong		< 2e-16 ***		
## districtSanxia		5.72e-10 ***		
## districtSanzhi		1.52e-10 ***		
## districtShinkeng		0.009172 **		
## districtShulin		0.000254 ***		
## districtTaishan		2.33e-12 ***		
## districtTamsui		0.204409		

```

## districtTucheng          5.14e-10 ***
## districtWugu             9.73e-07 ***
## districtXindian          < 2e-16 ***
## districtXinzhuang        < 2e-16 ***
## districtXizhi            < 2e-16 ***
## districtYingge           0.715567
## districtYonghe           < 2e-16 ***
## districtZhonghe          < 2e-16 ***
## buildingType             0.252109
## buildingType (10         ) 9.78e-06 ***
## parkingAreaL             0.197253
## balconyAreaL             0.980977
## landUseZoning            0.001253 **
## landUseZoning            0.308725
## landUseZoning            8.33e-10 ***
## landUseZoning            0.843261
## landUseZoning :          0.831664
## landUseZoning : ( )     0.009535 **
## landUseZoning :          7.88e-06 ***
## landUseZoning :          0.108339
## landUseZoning :          4.07e-09 ***
## landUseZoning :          0.326578
## landUseZoning :          0.477881
## parkingCategory          0.624682
## parkingCategory          0.682456
## parkingCategory          0.476457
## parkingCategory          0.050813 .
## parkingCategory          0.050587 .
## parkingCategory          0.789792
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2072 on 1157 degrees of freedom
## Multiple R-squared:  0.6647, Adjusted R-squared:  0.6523
## F-statistic: 53.35 on 43 and 1157 DF, p-value: < 2.2e-16

```



```

predictions <- predict(linearModel, test)

log_mse <- mean((Y_test - predictions)^2)
log_rmse <- rmse(Y_test, predictions)
log_mae <- mean(abs(Y_test - predictions))

mlr_metrics_log <- data.frame(
  Metric = c("MSE", "RMSE", "MAE"),
  Value = c(log_mse, log_rmse, log_mae)
) %>%
  mutate(Value = round(Value, 4))

kable(mlr_metrics_log, caption = "MLR Log Scale Performance")

```

MLR Log Scale Performance

Metric	Value
MSE	0.0394
RMSE	0.1985
MAE	0.1463

```

actual_prices    <- exp(Y_test)
predicted_prices <- exp(predictions)

mse  <- mean((actual_prices - predicted_prices)^2)
rmse_val <- rmse(actual_prices, predicted_prices)
mae_val <- mean(abs(actual_prices - predicted_prices))

mlr_metrics_exp <- data.frame(
  Metric = c("MSE", "RMSE", "MAE"),
  Value  = c(mse, rmse_val, mae_val)
) %>%
  mutate(Value = round(Value, 2))

kable(mlr_metrics_exp, caption = "MLR Original Scale Performance")

```

MLR Original Scale Performance

Metric	Value
MSE	378024840.51
RMSE	19442.86
MAE	14541.41

```

head(
  data.frame(
    Actual = actual_prices,
    Predicted = predicted_prices
  ),
  10
)

```

##	Actual	Predicted
## 1	60044	66185.07
## 2	69479	66451.48
## 3	68277	69452.35
## 4	70250	69452.35
## 5	70882	69452.35
## 6	69408	68300.54
## 7	95709	119157.30
## 8	112286	112747.12
## 9	131383	122316.64
## 10	116792	114656.44

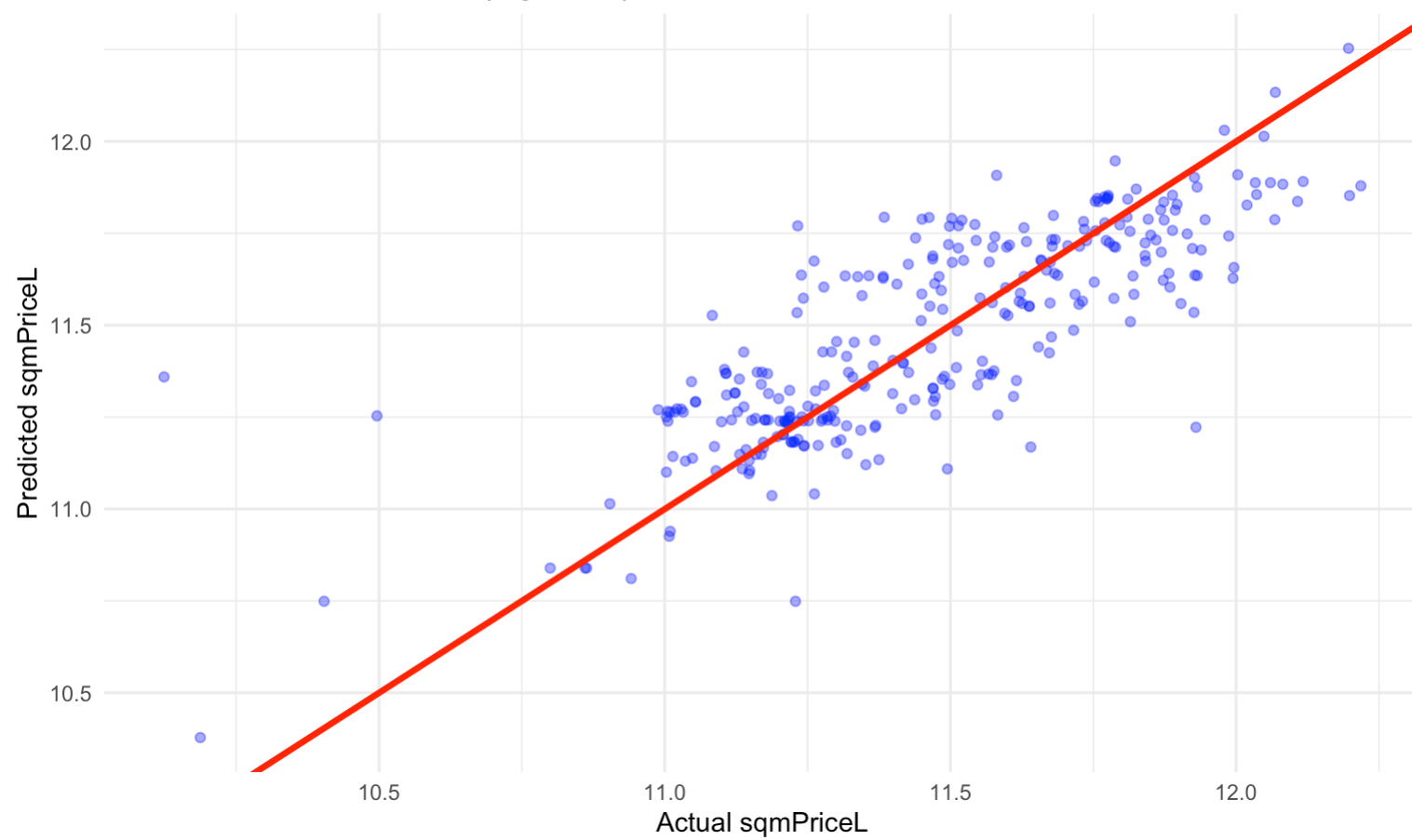
```
mlr_rmse <- rmse(actual_prices, predicted_prices)
mlr_r2    <- 1 - sum((actual_prices - predicted_prices)^2)/sum((actual_prices - mean(actual_prices))^2)

# =====
# Visualizations
# =====

# Predicted vs Actual (log scale)

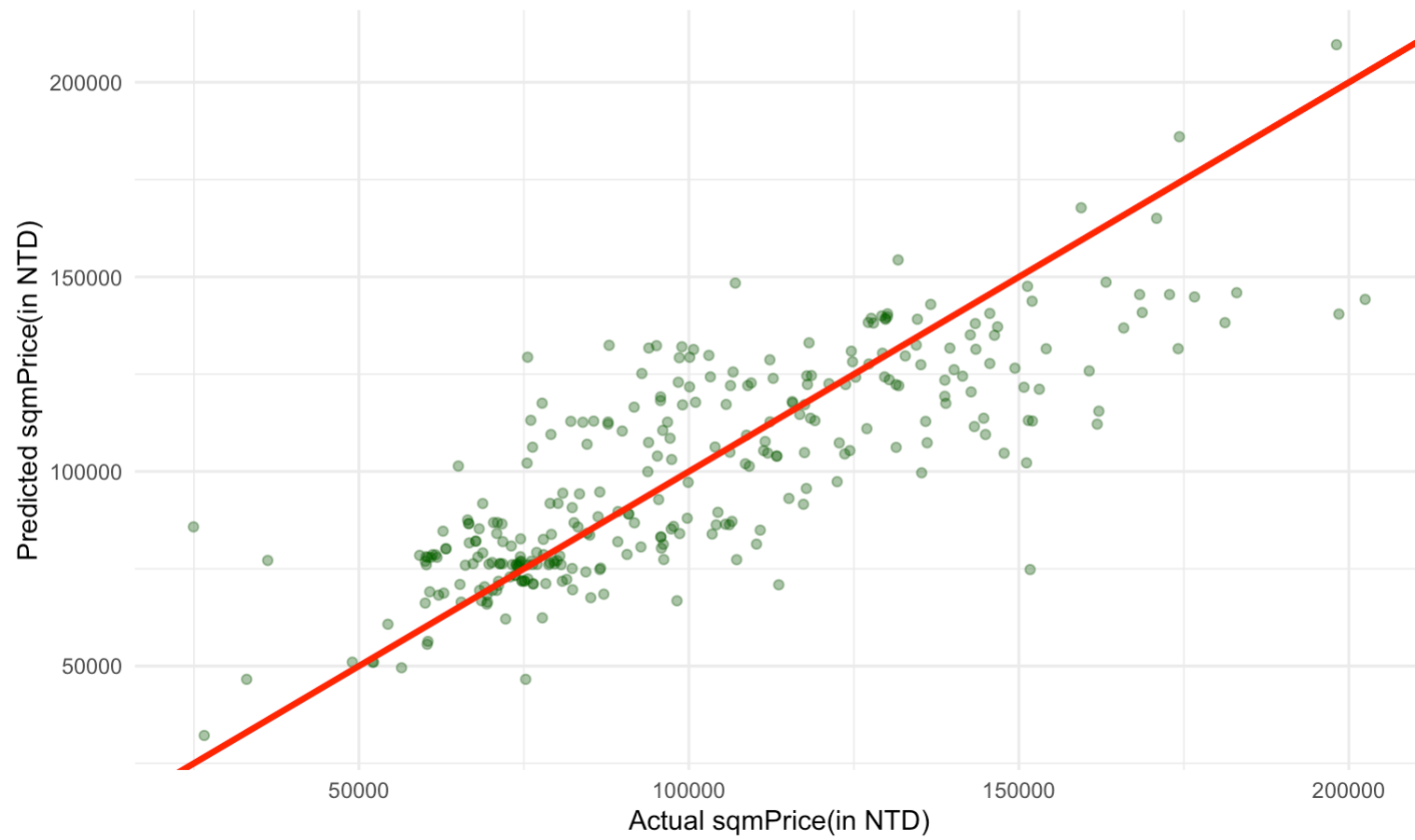
ggplot(data.frame(Actual = Y_test, Predicted = predictions), aes(x = Actual, y = Predicted)) +
  geom_point(color = "blue", alpha = 0.4) +
  geom_abline(slope = 1, intercept = 0, color = "red", size = 1.2) +
  labs(
    title = "MLR: Predicted vs Actual (log scale)",
    x = "Actual sqmPriceL",
    y = "Predicted sqmPriceL"
  ) +
  theme_minimal()
```

MLR: Predicted vs Actual (log scale)



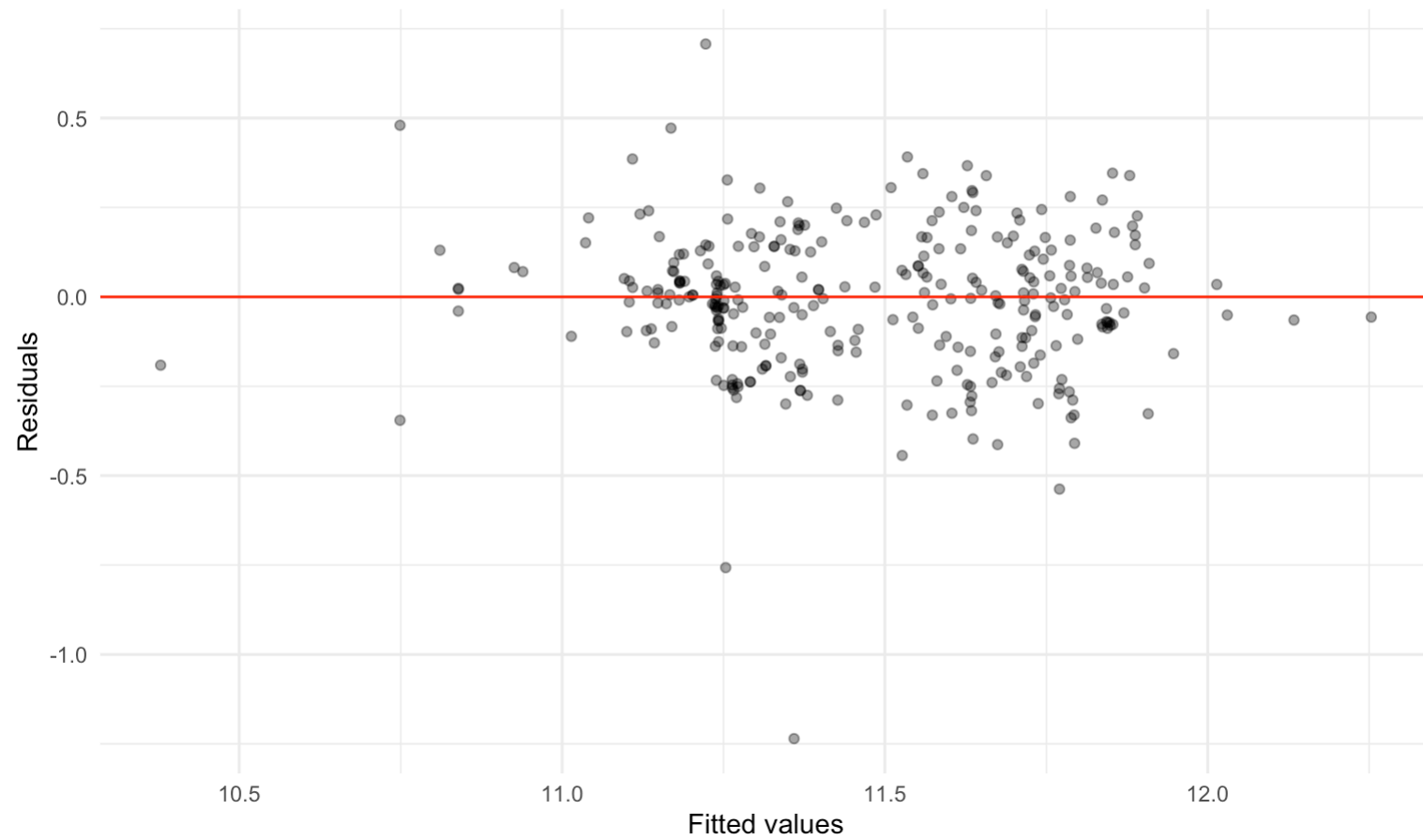
```
# Predicted vs Actual (original scale)
ggplot(data.frame(Actual = actual_prices, Predicted = predicted_prices), aes(x = Actual, y = Predicted))
  +
  geom_point(color = "darkgreen", alpha = 0.4) +
  geom_abline(slope = 1, intercept = 0, color = "red", size = 1.2) +
  labs(
    title = "MLR: Predicted vs Actual (original scale)",
    x = "Actual sqmPrice(in NTD)",
    y = "Predicted sqmPrice(in NTD)"
  ) +
  theme_minimal()
```

MLR: Predicted vs Actual (original scale)



```
# Residuals plot
residuals <- Y_test - predictions
ggplot(data.frame(Fitted = predictions, Residuals = residuals), aes(x = Fitted, y = Residuals)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, color = "red") +
  labs(
    title = "MLR: Residuals vs Fitted",
    x = "Fitted values",
    y = "Residuals"
  ) +
  theme_minimal()
```

MLR: Residuals vs Fitted



3.2 Random Forest

```
housing_X <- housing %>%  
  select(  
    sqmPriceL,  
    buildingAreaL,  
    age,  
    totalFloors,  
    district,  
    buildingType,  
    parkingAreaL,  
    balconyAreaL,  
    landUseZoning,  
    parkingCategory  
  ) %>%  
  filter(if_all(everything(), ~ !is.na(.) & !is.nan(.) & !is.infinite(.))) %>%  
  droplevels()  
  
smp_size <- floor(0.8 * nrow(housing_X))  
train_ind <- sample(seq_len(nrow(housing_X)), size = smp_size)  
  
train <- housing_X[train_ind, ]  
test  <- housing_X[-train_ind, ]  
  
Y_test <- test$sqmPriceL  
test$sqmPriceL <- NULL  
  
train_control <- trainControl(  
  method = "cv",  
  number = 3  
)  
  
mtry_grid <- expand_grid(  
  mtry = c(2, 4, 6)  
)
```

```
rf_tuned <- train(
  sqmPriceL ~ .,
  data = train,
  method = "rf",
  trControl = train_control,
  tuneGrid = mtry_grid,
  ntree = 300,
  importance = TRUE
)

print(rf_tuned)
```

```
## Random Forest
##
## 1201 samples
##    9 predictor
##
## No pre-processing
## Resampling: Cross-Validated (3 fold)
## Summary of sample sizes: 801, 801, 800
## Resampling results across tuning parameters:
##
##  mtry  RMSE      Rsquared  MAE
##  2     0.2653565  0.6254907  0.2041696
##  4     0.2164238  0.6723533  0.1518210
##  6     0.2005102  0.6930063  0.1317612
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 6.
```

```

predictions_rf <- predict(rf_tuned, test)

r2_log <- 1 - sum((Y_test - predictions_rf)^2) /
  sum((Y_test - mean(Y_test))^2)

rf_metrics <- data.frame(
  Metric = c("MAE", "MSE", "RMSE", "R²"),
  Value = c(
    mae(Y_test, predictions_rf),
    mse(Y_test, predictions_rf),
    rmse(Y_test, predictions_rf),
    r2_log
  )
) %>%
  mutate(Value = round(Value, 2))

kable(rf_metrics, caption = "Random Forest – Log Scale Performance")

```

Random Forest – Log Scale Performance

Metric	Value
MAE	0.11
MSE	0.03
RMSE	0.16
R ²	0.78

```

actual_prices <- exp(Y_test)
predicted_prices <- exp(predictions_rf)

r2_exp <- 1 - sum((actual_prices - predicted_prices)^2) /
  sum((actual_prices - mean(actual_prices))^2)

rf_metrics_exp <- data.frame(
  Metric = c("MAE", "MSE", "RMSE", "R²"),
  Value = c(
    mae(actual_prices, predicted_prices),
    mse(actual_prices, predicted_prices),
    rmse(actual_prices, predicted_prices),
    r2_exp
  )
) %>%
  mutate(Value = round(Value, 2)) # Round to 2 decimals

kable(rf_metrics_exp, caption = "Random Forest – Original Scale Performance")

```

Random Forest – Original Scale Performance

Metric	Value
MAE	11847.93
MSE	298518415.91
RMSE	17277.69
R²	0.77

```

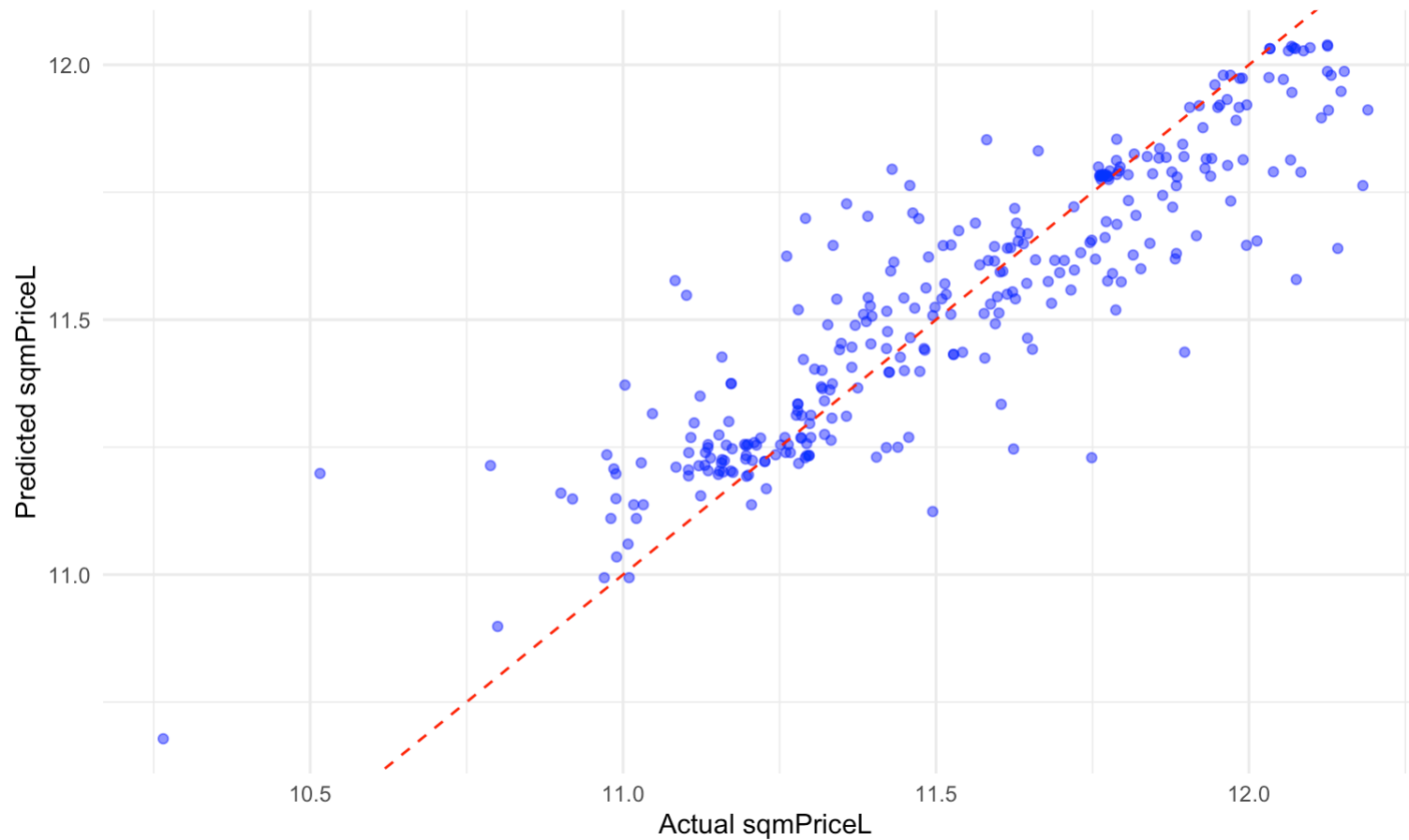
rf_rmse <- rmse(actual_prices, predicted_prices)
rf_r2    <- 1 - sum((actual_prices - predicted_prices)^2)/sum((actual_prices - mean(actual_prices))^2)

# -----
# Visualizations
# -----

# Predicted vs Actual (log scale)
ggplot(data = data.frame(Actual = Y_test, Predicted = predictions_rf), aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.5, color = "blue") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  labs(title = "Random Forest: Predicted vs Actual (log scale)",
       x = "Actual sqmPriceL",
       y = "Predicted sqmPriceL") +
  theme_minimal()

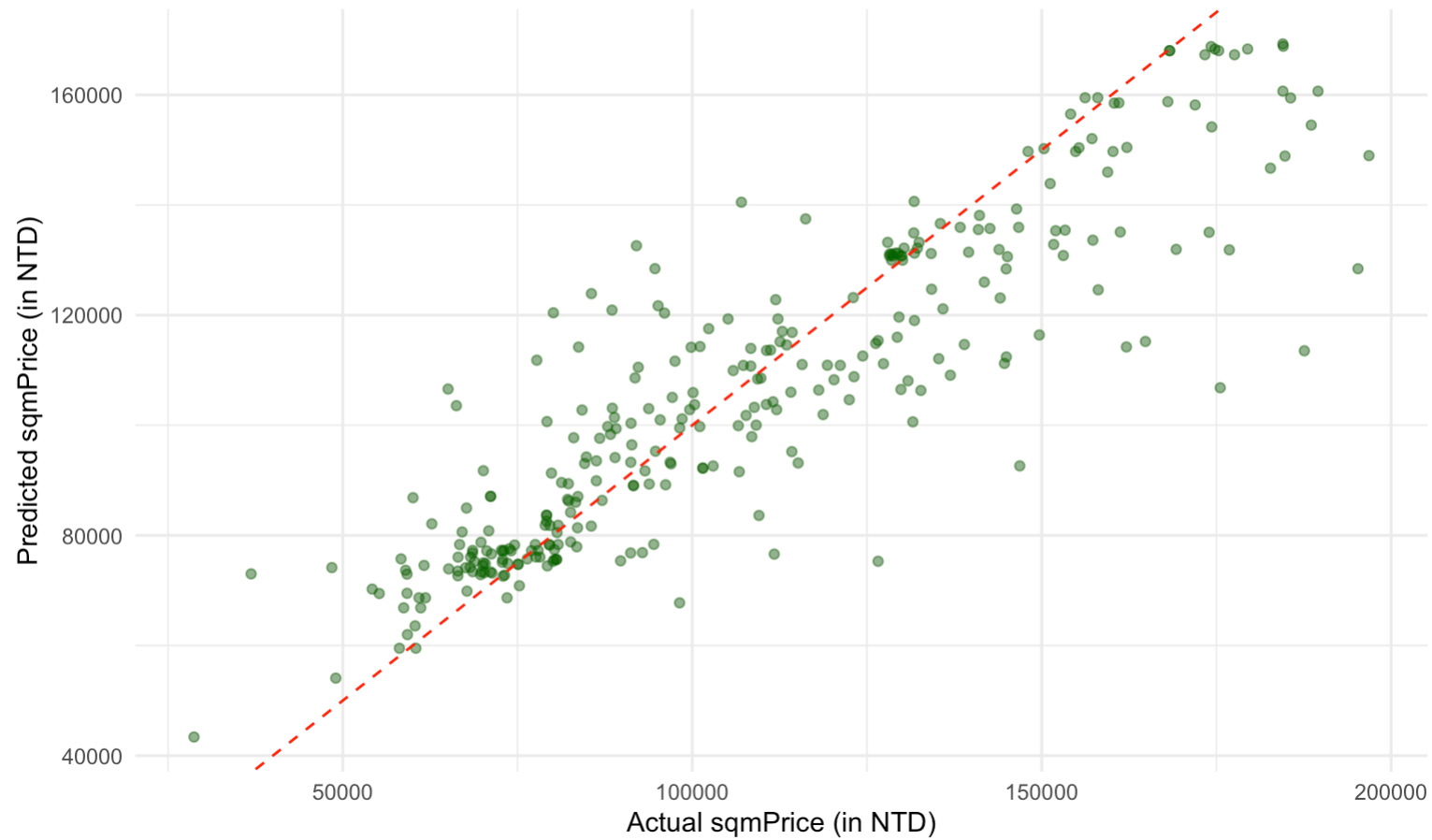
```

Random Forest: Predicted vs Actual (log scale)



```
# Predicted vs Actual (original scale)
ggplot(data = data.frame(Actual = actual_prices, Predicted = predicted_prices), aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.5, color = "darkgreen") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  labs(title = "Random Forest: Predicted vs Actual (original scale)",
        x = "Actual sqmPrice (in NTD)",
        y = "Predicted sqmPrice (in NTD)") +
  theme_minimal()
```

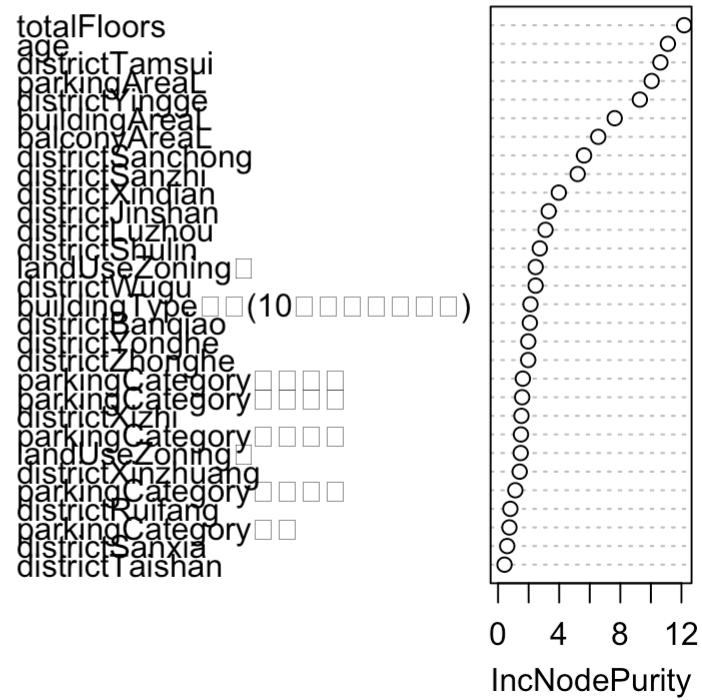
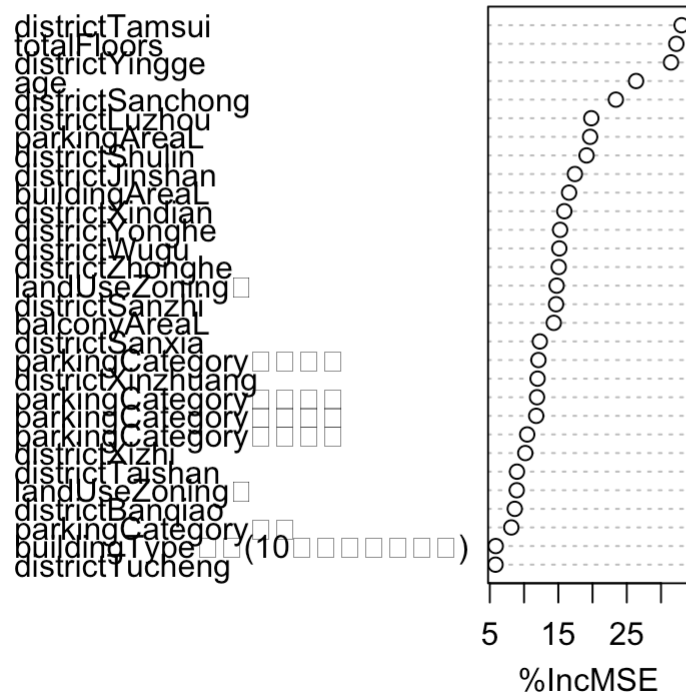
Random Forest: Predicted vs Actual (original scale)



Variable Importance

```
varImpPlot(rf_tuned$finalModel, main = "Random Forest Variable Importance")
```

Random Forest Variable Importance



3.3 XGBoost

```
housing_X <- housing %>%
  select(
    sqmPriceL,
    buildingAreaL,
    age,
    totalFloors,
    district,
    buildingType,
    parkingAreaL,
    balconyAreaL,
    landUseZoning,
    parkingCategory
  ) %>%
  filter(if_all(everything(), ~ !is.na(.) & !is.nan(.) & !is.infinite())) %>%
  droplevels()

dummy_vars <- dummyVars(sqmPriceL ~ ., data = housing_X)
housing_matrix <- predict(dummy_vars, housing_X) %>% as.matrix()

Y <- housing_X$sqmPriceL

smp_size <- floor(0.8 * nrow(housing_matrix))
train_ind <- sample(seq_len(nrow(housing_matrix)), size = smp_size)

X_train <- housing_matrix[train_ind, ]
X_test  <- housing_matrix[-train_ind, ]

Y_train <- Y[train_ind]
Y_test  <- Y[-train_ind]

dtrain <- xgb.DMatrix(data = X_train, label = Y_train)
dtest  <- xgb.DMatrix(data = X_test)
```

```

params <- list(
  objective = "reg:squarederror",
  eval_metric = "rmse",
  eta = 0.05,
  max_depth = 6,
  subsample = 0.8,
  colsample_bytree = 0.8
)

xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 300,
  verbose = 0
)

predictions_xgb <- predict(xgb_model, dtest)

r2_log <- 1 - sum((Y_test - predictions_xgb)^2) /
  sum((Y_test - mean(Y_test))^2)

xgb_metrics_log <- data.frame(
  Metric = c("MAE", "MSE", "RMSE", "R²"),
  Value = c(
    mae(Y_test, predictions_xgb),
    mse(Y_test, predictions_xgb),
    rmse(Y_test, predictions_xgb),
    r2_log
  )
) %>%
  mutate(Value = round(Value, 2))

kable(xgb_metrics_log, caption = "XGBoost – Log Scale Performance")

```

XGBoost – Log Scale Performance

Metric	Value
MAE	0.12
MSE	0.04
RMSE	0.19
R ²	0.73

```
actual_prices <- exp(Y_test)
predicted_prices <- exp(predictions_xgb)

r2_exp <- 1 - sum((actual_prices - predicted_prices)^2) /
  sum((actual_prices - mean(actual_prices))^2)

xgb_metrics_exp <- data.frame(
  Metric = c("MAE", "MSE", "RMSE", "R²"),
  Value = c(
    mae(actual_prices, predicted_prices),
    mse(actual_prices, predicted_prices),
    rmse(actual_prices, predicted_prices),
    r2_exp
  )
) %>%
  mutate(Value = round(Value, 2))

kable(xgb_metrics_exp, caption = "XGBoost – Original Scale Performance")
```

XGBoost – Original Scale Performance

Metric	Value
--------	-------

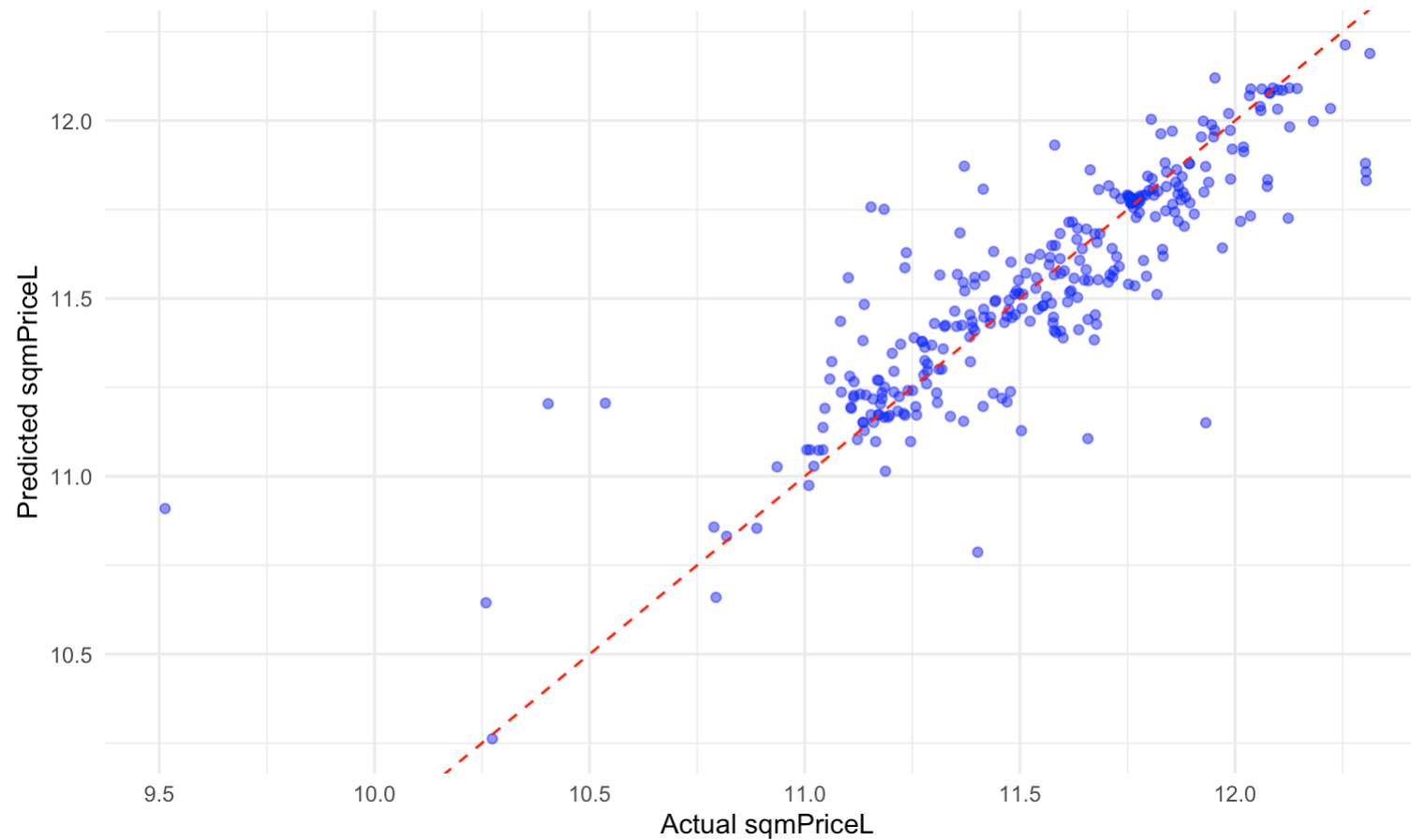
Metric	Value
MAE	12038.19
MSE	346271956.24
RMSE	18608.38
R^2	0.75

```
xgb_rmse <- rmse(actual_prices, predicted_prices)
xgb_r2    <- 1 - sum((actual_prices - predicted_prices)^2)/sum((actual_prices - mean(actual_prices))^2)

# -----
# Visualizations
# -----

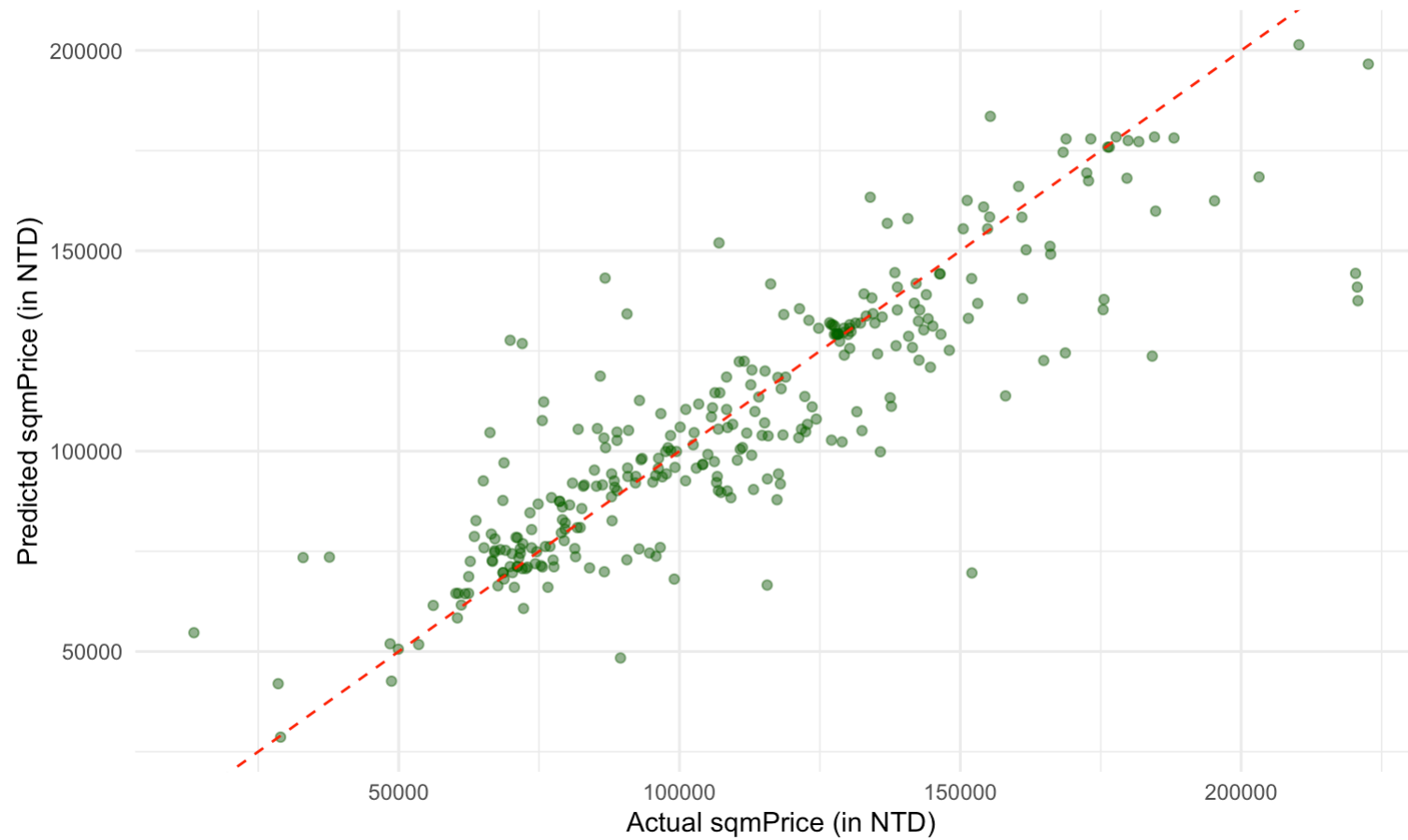
# Predicted vs Actual (log scale)
ggplot(data = data.frame(Actual = Y_test, Predicted = predictions_xgb), aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.5, color = "blue") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  labs(title = "XGBoost: Predicted vs Actual (log scale)",
       x = "Actual sqmPriceL",
       y = "Predicted sqmPriceL") +
  theme_minimal()
```

XGBoost: Predicted vs Actual (log scale)



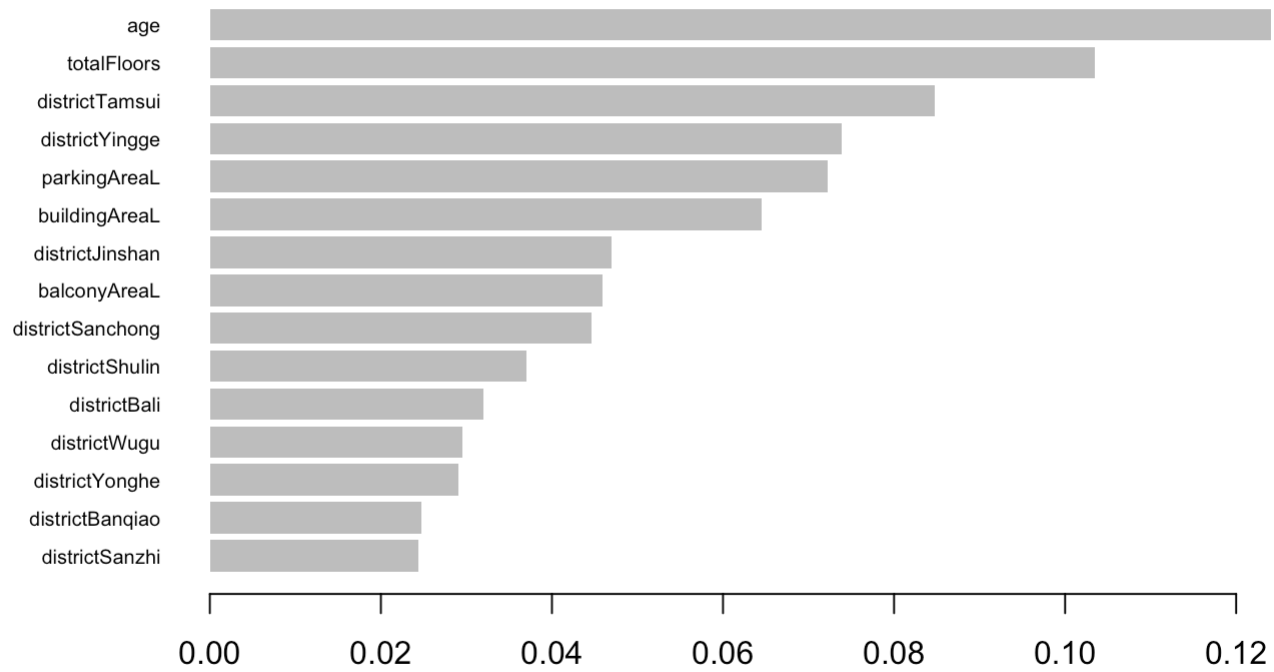
```
# Predicted vs Actual (original scale)
ggplot(data = data.frame(Actual = actual_prices, Predicted = predicted_prices), aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.5, color = "darkgreen") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  labs(title = "XGBoost: Predicted vs Actual (original scale)",
        x = "Actual sqmPrice (in NTD)",
        y = "Predicted sqmPrice (in NTD)") +
  theme_minimal()
```

XGBoost: Predicted vs Actual (original scale)



```
# Feature Importance
xgb_imp <- xgb.importance(model = xgb_model)
xgb.plot.importance(xgb_imp, top_n = 15, main = "XGBoost Feature Importance")
```

XGBoost Feature Importance



4 Model Comparison

```

metrics_table <- data.frame(
  Model = c("MLR","Random Forest","XGBoost"),
  RMSE = c(mlr_rmse, rf_rmse, xgb_rmse),
  R2 = c(mlr_r2, rf_r2, xgb_r2)
)
metrics_table <- metrics_table %>% mutate(across(where(is.numeric), ~ round(.,2)))
kable(metrics_table, caption="Comparison of RMSE and R2 Across Models") %>% kable_styling(full_width=F, position="center", bootstrap_options=c("striped","hover"))

```

Comparison of RMSE and R² Across Models

Model	RMSE	R2
MLR	19442.86	0.65
Random Forest	17277.69	0.77
XGBoost	18608.38	0.75